Syrian Private University

Factuly of Artificial Intelligent Engineering

System and Network Security Engineering

Cyber Attack Analysis Using Honeypots and Knowledge Graphs

Prepared by :

Mayas Alrefaie **Masa Daboul**

Supervised by :

Dr .Waseem Junaidi

Eng .Ahmad Sheikhah

March 2026

Introduction

1.Introduction

Cybersecurity stands as one of the most pressing challenges of the digital age, as attack methodologies evolve at a pace that outstrips the capacity of traditional defense systems to respond effectively. Honeypots constitute one of the most valuable instruments for studying attacker behavior within a controlled environment, functioning as decoy systems that lure attacks and collect precise data on attackers' tools, techniques, and motivations. In recent years, Large Language Models (LLMs) have emerged as a technology capable of fundamentally transforming honeypot design by enabling dynamic, intelligent interaction with attackers that transcends the limitations of static responses. In parallel, Knowledge Graphs (KGs) have demonstrated their efficacy in representing semantic relationships among heterogeneous security entities, thereby facilitating deeper analysis and revealing patterns that remain obscured in raw data. This project seeks to integrate these three components an SSH honeypot, an LLM-based honeypot, and a knowledge graph into a unified framework for cyber attack analysis using real-world data collected from actual attackers.

2.ResearchProblem

Traditional honeypots suffer from a structural deficiency in attacker interaction, as they produce static and easily detectable responses, which constrains both the depth and the quality of the collected data. Furthermore, most attack analysis systems process events in isolation from their context, failing to correlate attackers, tools, infrastructure, and techniques within a unified relational model. Additionally, there exists no clear scientific methodology for transforming raw honeypot data into machine-inferable knowledge. These limitations are exacerbated by the evolving nature of attack methodologies, which are characterized by distribution, stealth, and adaptability, rendering static solutions insufficient. Consequently, a research gap emerges for a model that integrates intelligent interaction with structured knowledge representation to bridge this divide.

3.ProjectObjectives

The project aims to design and implement an integrated research framework for deploying an SSH honeypot on a Virtual Private Server (VPS), configuring it to collect real world attack data from actual attackers, and subsequently analyzing and representing this data within a knowledge graph that captures the relationships among security entities. It further aims to explore LLM enhanced honeypots as an advanced generation of decoy systems capable of intelligent interaction with attackers and the generation of contextually realistic responses, while assessing the impact thereof on the quality and analytical value of the collected data. Ultimately, the project seeks to deliver a reproducible and extensible methodology that contributes to research in cyber attack analysis and threat intelligence.

4. Practical Applications

The project's outcomes manifest across several vital application domains. At the defensive level, the collected data and extracted knowledge relationships can be leveraged to enhance intrusion detection systems and cyber threat intelligence platforms by providing contextually rich threat indicators. At the response level, the knowledge graph supports the construction of automated response systems capable of inferring attack chains and making decisions based on relationships rather than isolated events. Moreover, the project benefits security team training through realistic attack scenarios and advances research in malicious behavior analysis and attacker motivation modeling. Additionally, it provides a reference framework for researchers seeking to integrate LLMs with knowledge graphs in security applications.

5. Challenges

The project faces multifaceted challenges. At the security level, it is imperative to ensure the isolation of the honeypot environment and prevent its exploitation as a launchpad for subsequent attacks, while preserving the confidentiality of the underlying infrastructure. At the technical level, designing an LLM-based honeypot capable of maintaining contextual coherence and concealing its artificial nature represents a delicate challenge that necessitates rigorous control over performance and resource consumption. Furthermore, constructing the knowledge graph requires the selection of appropriate ontologies and effective mechanisms for entity and relation extraction from unstructured data. At the methodological level, the challenge of evaluating and calibrating the quality of the collected data is particularly salient.

Chapter 1 : Basic Concepts

1-1.Introduction

This chapter addresses the fundamental theoretical concepts underpinning the project, which were derived from contemporary scientific literature in the fields of honeypots, large language models, and knowledge graphs. It aims to establish a clear conceptual foundation that facilitates the comprehension of the project's methodology and findings in the subsequent chapters.

2-1.Honeypot

A honeypot is a decoy system designed to entice and deceive adversaries, luring them away from genuine assets to collect threat intelligence regarding their tools, techniques, and motivations. Honeypots are classified by interaction level into three categories: low-interaction, which emulates simple services; medium-interaction, which provides a simulated shell; and high-interaction, which relies on real systems exposed to compromise. Higher interaction levels offer greater deception capability against sophisticated attackers but increase management complexity and the risk of misuse [11].

3-1.SSH-Honeypot

An SSH honeypot is a decoy system that emulates a Secure Shell (SSH) service on a server, causing the attacker to believe they are interacting with a genuine, exploitable system. These honeypots are primarily used to detect brute-force attacks against SSH/Telnet services, logging all authentication attempts and executed commands. Cowrie is one of the most well-known examples of medium-interaction honeypots that emulate a Unix shell with a predefined file system [12].

4-1.Large-Language-Model(LLM)

A Large Language Model (LLM) is an AI model based on transformer architecture, trained on massive textual corpora, and capable of understanding complex contexts and generating coherent text, program code, and structured outputs. Model performance on complex tasks generally correlates with model size. These models have become central to security automation

applications, including vulnerability analysis, penetration testing automation, and network service simulation [11].

5-1.LLM-Based-Honeypot

An LLM-based honeypot is an advanced generation of decoys that leverages a large language model to generate dynamic, realistic responses to attacker commands in real time. Rather than relying on static pre-scripted responses, the model interprets commands and generates contextually appropriate outputs that mimic a genuine Linux shell, thereby enhancing the honeypot's credibility and prolonging attacker engagement. The balance between authenticity and performance is achieved by integrating a cache of common commands with LLM invocation for novel inputs [13].

6-1.Knowledge-Graph

A Knowledge Graph is a data structure that represents knowledge as entities and semantic relations between them, enabling structured representation and machine-inferable knowledge. In cybersecurity, it is used to link heterogeneous security entities such as attackers, tools, infrastructure, and techniques within a unified model that supports complex queries and deeper pattern analysis. Its construction involves data collection, preprocessing, graph transformation, and subsequent analysis [14].

7-1. Prompt Injection

Prompt injection is an attack technique that involves inserting carefully crafted instructions into a language model's input to manipulate its behavior and steer it toward specific goals. In LLM-enhanced honeypots, this technique is employed as an active mechanism to distinguish LLM agents from human attackers by testing their response to instructions such as "ignore previous instructions" or "Important Message," while measuring response time to confirm the automated nature of the interaction [15].

8-1.Virtual-File-System

A Virtual File System is a software component in medium- to high-interaction honeypots that emulates a real system's file structure without running an actual operating system. It aims to maintain system state consistency across successive commands, such that if an attacker creates a file and later attempts to read it, the file persists. This component prevents the language model

from contradicting its earlier outputs, thereby reducing hallucination and enhancing honeypot credibility [13].

9-1.ResponseLatency

Response latency is the time interval between a honeypot receiving an attacker's command and dispatching the ready output. In LLM-enhanced honeypots, response latency is a critical factor in maintaining interaction credibility, as excessive delay may arouse the attacker's suspicion. Experimental studies show that most medium-sized models exhibit latencies between 2 and 4 seconds — a range that may be noticeable but not unrealistic — while some models exceed 10 seconds, rendering them impractical for live interaction [13].

10-1.Brute-Force-Attack

A brute-force attack is an offensive technique that relies on the repeated trial of all possible combinations of passwords or cryptographic keys until the correct one is found. In the context of SSH services, this is the most prevalent attack type, with botnets attempting to compromise accounts using lists of common passwords and typical usernames such as root and admin. SSH honeypots are deployed to detect and log such attacks [12].

11-1.Botnet

A botnet is a network of compromised devices controlled remotely by an attacker, typically used to launch large-scale attacks such as brute-force attempts, denial-of-service, and malware distribution. In the context of honeypots, botnets constitute the majority of attack traffic, as automated bots continuously scan for vulnerable services and attempt exploitation. Honeypot studies frequently report that within minutes of deployment, they log over a thousand attacks predominantly originating from botnets [12].

12-1.Threat-Intelligence

Threat intelligence is organized and analyzed knowledge concerning cyber threats, including information about attackers, their tools, techniques, infrastructure, and targets. Honeypots constitute a vital source of such intelligence, providing real world data on actual attacks. The

volume and quality of collected intelligence are measured by criteria such as attack frequency, diversity, interaction duration, and volume of captured data [11].

13-1.Behavioral-Analysis

Behavioral analysis, in the context of honeypots, is a methodology aimed at studying patterns of attacker interaction with the decoy system, including command sequences, inter-command timing, and exploration paths. This analysis enables the extraction of indicators regarding the attacker's skill level, motivations, and tools. When integrated with knowledge graphs, the analysis becomes capable of correlating disparate sessions and identifying recurring attack patterns across different IP addresses [14].

14-1.Stateful-Interaction

Stateful interaction is a honeypot's ability to maintain system state across an attacker's session or between multiple sessions, such that files, directories, and users created by the attacker persist upon reconnection. This concept is a critical factor in honeypot credibility, as genuine systems retain their state, whereas many traditional honeypots fail to do so. Stateful interaction is achieved by passing the previous conversation history to the language model as context upon reconnection [16].

15-1.Honeypot-Fingerprinting

Honeypot fingerprinting is the process of determining whether a system is a honeypot or a genuine system by analyzing its responses to specific query patterns. Techniques include measuring response time, sending unconventional commands such as pinging invalid addresses, and analyzing technical details that may reveal the system's virtual nature (e.g., CPU type or virtualization environment).

Fingerprint resistance is a fundamental challenge in honeypot design, as detection negates their defensive value [11].

16-1.Attack-Surface

Attack surface, in the context of honeypots, is the set of interaction points, services, ports, and vulnerabilities available to an attacker within the decoy system. The broader the attack surface and the more simulated services, the higher the honeypot's credibility and attractiveness to attackers. An effective attack surface includes the availability of package management tools, file

upload and script execution capabilities, and the simulation of known vulnerabilities (such as kernel exploits) that give the attacker a sense of genuine exploitation opportunities [11].

17-1.Agent

An agent is a software system capable of making decisions and executing actions autonomously within a given environment, based on its perception of state and its goals. In cybersecurity, recent research points to the emergence of LLM-powered hacking agents capable of adapting to diverse environments and executing complex attack behaviors. The agent concept is also employed in building intelligent honeypot systems that interact automatically with attackers [15].

18-1.Deception-Framework

A deception framework is an integrated system that combines a set of interconnected decoy services and components such as SSH, MySQL, HTTP, and POP3 simulations within a unified, deployable, and manageable architecture. The framework aims to provide a multi-layered deception environment that is harder to detect and prolongs attacker engagement. It allows users to select which services to simulate based on their defensive needs [16].

19-1.Dwell-Time

Dwell time is the duration an attacker remains engaged with a honeypot before disconnecting or detecting its deceptive nature. This duration is a key indicator of honeypot effectiveness, as the volume of collected threat intelligence increases proportionally with dwell time. Studies indicate that LLM-enhanced honeypots are capable of prolonging dwell time compared to traditional honeypots, owing to their dynamic and realistic responses that sustain attacker engagement [13]

20-1.Cybersecurity

Cybersecurity is the set of practices, technologies, and controls designed to protect systems, networks, software, and data from digital attacks aimed at unauthorized access, alteration, destruction, or exfiltration. Cybersecurity encompasses multiple domains including network security, application security, information security, and operational security, and requires a multi-layered approach combining preventive, detective, and corrective measures. In the context of the present research, cybersecurity represents the broader domain to which the project belongs [11].

21-1.CyberThreat

A cyber threat is any circumstance, event, or action with the potential to cause harm to systems, networks, or data, whether intentionally by an attacker or unintentionally. Cyber threats include malware, denial-of-service attacks, phishing, brute-force attacks, and vulnerability exploitation. Understanding the nature of a threat is the first step in designing effective defense mechanisms, as each threat type requires a different response [11].

22-1.Virtual-Private-Server(VPS)

A Virtual Private Server (VPS) is a virtualized computing environment running on a single physical server, characterized by dedicated resources (CPU, memory, storage) and isolation from other virtual environments. VPS is widely used in honeypot deployment due to its lower cost compared to physical servers, ease of control, and ability to simulate a real server without exposing actual infrastructure to risk. VPS represents the optimal choice for running SSH honeypots in a production environment [12].

23-1.AttackerBehavior

Attacker behavior is the set of actions and decisions made by an individual or system during an attempt to compromise a target, encompassing phases of reconnaissance, scanning, exploitation, escalation, and persistence. Analyzing attacker behavior is a fundamental objective of honeypots, as it enables understanding of the attacker's motivations, skill level, and tools, thereby contributing to the construction of proactive defense systems capable of predicting future attacks [14].

24-1.Session

A session is the time interval extending from the initiation of an attacker's connection to the honeypot until the termination of that connection, encompassing all commands executed and outputs generated during that period. The session constitutes the fundamental unit for attacker behavior analysis, as each session is recorded with a unique identifier, timestamp, IP address, and command set. Session analysis contributes to identifying attack patterns and classifying attackers according to their threat level [14].

25-1.Command-Sequence

A command sequence is the chronological ordering of commands executed by an attacker within a single session or across multiple sessions, serving as a fundamental indicator of the attacker's intent and strategy. Analyzing command sequences enables the extraction of recurring attack patterns, such as privilege escalation attempts, sensitive data exfiltration, or malware installation. When command sequences are linked to a knowledge graph, it becomes possible to uncover relationships between different sessions and attribute them to a single attacker [14].

26-1.IP Address Attribution

IP address attribution is the process of linking observed malicious activity to a specific IP address as the attack source, involving the collection and analysis of all sessions and commands originating from that address. This process is employed in building geographic maps of attackers, identifying recurring patterns, and classifying threat levels. Honeypots provide accurate data for this type of analysis, as every incoming connection is logged with its source address [14].

27-1.Active-Inference

Active Inference is a theoretical framework describing intelligent system behavior as a Bayesian inference process aimed at reducing uncertainty in its internal model of the environment. The framework is grounded in the Free Energy Principle, whereby the agent seeks to minimize Bayesian surprise through two levers: acting on the environment and updating the internal model. In active cyber-defense, this framework is employed to guide the agent toward actions that reveal new information about threats [17].

28-1.Prompt-Engineering

Prompt engineering is the process of designing and crafting instructions (prompts) presented to a large language model to steer its behavior toward achieving a specific goal. This process is critical in LLM-enhanced honeypot applications, as the quality of designed prompts determines the realism of the model's responses and its ability to simulate a genuine system. Techniques employed in prompt engineering include chain-of-thought reasoning, step-by-step reasoning, and providing examples [16].

29-1.Ontology

Ontology, in the context of knowledge graphs, is a formal and structured representation of concepts and relations within a specific knowledge domain, defining the classes, properties, and relations governing entities in that domain. In cybersecurity, ontologies are used to ensure unified and consistent representation of security entities such as attackers, vulnerabilities, and techniques, thereby facilitating querying and machine inference. Selecting the appropriate ontology is a critical step in constructing an effective knowledge graph [14].

30-1.Entity-Extraction

Entity extraction is the process of identifying and extracting meaningful elements (such as usernames, IP addresses, filenames, and commands) from unstructured data, with the aim of transforming them into entities representable in a knowledge graph. This process is a fundamental step in constructing knowledge graphs from honeypot data, as it enables the conversion of raw textual logs into structured knowledge amenable to querying and analysis. The effectiveness of extraction depends on the quality of source data and the accuracy of processing mechanisms [14].

31-1. Relation Extraction

Relation extraction is the process of identifying semantic links between entities extracted from data, such as the "used" relation between an attacker and a tool, or the "targeted" relation between an attacker and an IP address. This process complements entity extraction in constructing knowledge graphs, as it enables representation of interactions among different elementes.

32-1.Deception

Deception in cybersecurity is a defensive approach aimed at misleading the attacker by presenting false information, systems, or services, with the objective of slowing the attack, diverting the attacker's attention, or gathering intelligence about them. Deception constitutes an active defensive strategy that transcends passive protection, as it does not merely prevent attacks but seeks to entice and study the attacker. Honeypots represent one of the most prominent applications of deception in cybersecurity [16].

33-1.Multi-Layered-Deception

Multi-layered deception is a defensive strategy that combines multiple interconnected decoy services or components (such as SSH, MySQL, and HTTP) within a unified architecture, aiming to increase the difficulty of detecting the deceptive environment and prolonging attacker engagement. This strategy allows the attacker to transition from one service to another, thereby providing greater opportunities to observe their behavior and collect data. This approach is more effective than relying on a single decoy service [16].

34-1.Sandbox

A sandbox is a restricted execution environment in which processes or applications are isolated from the actual system, with the aim of preventing any harmful impact on the infrastructure. In the context of LLM-enhanced honeypots, all attacker commands are executed within an isolated environment that does not reach a genuine operating system, thereby preventing misuse of the honeypot. The sandbox constitutes an essential element in designing secure honeypots [16].

35-1.Context-Window

A context window is the maximum number of tokens a language model can process at one time, encompassing both input instructions and generated outputs. In LLM-enhanced honeypot applications, the context window determines the amount of session history that can be passed to the model to maintain consistency. When the session history exceeds the window size, the older portion is discarded, potentially leading to loss of consistency with previous sessions [16].

36-1.Fine-Tuning

Fine-tuning is an additional training process for a pre-trained language model on a specialized dataset, aiming to adapt it to a specific task. In the context of LLM-enhanced honeypots, fine-tuning is used to train the model to simulate a Linux shell with higher accuracy, thereby reducing hallucination and improving response quality. Fine-tuning is more costly than relying on prompt engineering alone, but it yields more precise results [16].

37-1.Conclusion

This chapter presented the fundamental theoretical concepts underpinning the project, encompassing the definition of cybersecurity and cyber threats, and honeypots in their three types (low-, medium-, and high-interaction), along with SSH honeypots as the pivotal instrument for collecting real-world attack data. The chapter also addressed large language models and their applications in designing intelligent honeypots, along with associated concepts such as prompt engineering, fine-tuning, context window, hallucination, and response latency. Furthermore, the chapter reviewed knowledge graphs as a tool for representing semantic relationships among security entities, with related concepts such as ontology, entity and relation extraction, and behavioral analysis. Finally, the chapter addressed concepts related to active defense, including Markov Decision Processes, Active Inference, reinforcement learning, and epistemic gain.

Chapter 2 : Study References

1-2.Introduction

This chapter presents a comprehensive review of previous studies related to cyber attack analysis, honeypot technologies, cybersecurity logs, Cyber Threat Intelligence, and Knowledge Graphs. It aims to examine existing approaches for collecting, analyzing, representing, and interpreting cyber attack data, while identifying their main achievements, features, and limitations. Previous research has demonstrated that honeypots are effective tools for attracting malicious activities and collecting realistic information about attackers, including their IP addresses, targeted services, executed commands, attack techniques, and behavioral patterns. However, the large volume and heterogeneous nature of honeypot-generated logs make manual analysis difficult and time-consuming. Recent studies have therefore explored Knowledge Graphs as a means of representing cybersecurity information through entities and relationships, enabling security analysts to understand the relationships between individual events and broader attack patterns.

2-2. Previous Studies

Table (1) reviewed ten reference studies published exclusively during 2025 and 2026, ordered from most recent to oldest. These studies constitute the scientific foundation upon which the present project is built, covering three main axes: LLM-enhanced honeypots, Knowledge Graph construction from security logs, and mapping attacks to MITRE ATT&CK.
In the axis of honeypots and Large Language Models, the OntoLogX study [1] proposed a framework for converting raw security logs into structured Knowledge Graphs using LLMs, linking low-level evidence with MITRE ATT&CK tactics, providing a clear pipeline: Honeypot → Logs → Knowledge Graph → Techniques. Similarly, the CTI-Thinker system [2] built a CTI Knowledge Graph with attack reasoning support, enabling a transition from simple representation like "IP executed a command" to a richer one like "IP linked to behavior indicating a specific technique." Oliveira et al. [3] proposed an integrated framework combining a Cowrie honeypot with a local LLM, achieving Accuracy = 94.23%, Precision = 100%, F1 = 93.33%, with detection within 25 seconds. The Q-Cowrie study [4] added an adaptive dimension using Reinforcement Learning and Markov Decision Processes. Alshehri [5] proposed a dynamic deception framework achieving 98% detection accuracy and 95% deception effectiveness, defining a future direction: collect → analyze→understand→adapt.

In the axis of Knowledge Graphs and CTI analysis, Yilmaz et al. [6] used Graph Neural Networks for anomaly detection and visualization of CTI-based Knowledge Graphs, achieving GraphSAGE F1 = 0.9721 and GIN F1 = 0.9693. Au et al. [7] built a heterogeneous Knowledge Graph from multi-source CTI to link low-level alerts with high-level APT information, linking: Attack → Technique → Threat Actor → Campaign. Li et al. [8] used a Knowledge Graph as an additional view to improve intrusion detection (F1 = 0.963 on TON_IoT). Nair et al. [9] analyzed over 690,000 attacks using T-Pot, justifying the use of honeypots, while KGRAG [10] added the concept of provenance to assess information reliability, identifying threat actors with 95% accuracy.

Based on the above, it becomes clear that previous studies have made substantial contributions in each of the three axes separately, but have not combined them into a unified framework that directly transforms honeypot data into an analyzable Knowledge Graph. This is the gap addressed by the present project.

Table 1: Study References

Ref.	Article Title	Source	Authors	Year	Main Findings	Relevance to Our Project
[1]	OntoLogX: Ontology-Guided Knowledge Graph Extraction From Cybersecurity Logs With Large Language Models	Advanced Intelligent Systems (Wiley)	Cotti et al.	2026	Converts raw logs into structured KG and links them with MITRE ATT&CK	Provides a clear pipeline: Honeypot → Logs → KG → Techniques
[2]	CTI-Thinker: An LLM-Driven System for CTI Knowledge Graph Construction and Attack Reasoning	Cybersecurity (Springer)	. Yang et al	2026	Builds a CTI Knowledge Graph and supports attack reasoning	Justifies moving from "IP executed command" to "IP linked to attack technique

Ref.	Article Title	Source	Authors	Year	Main Findings	Relevance to Our Project
[3]	An Integrated Honeypot and LLM-Based Framework for Near Real-Time Detection and Behavioral Analysis of Malicious Activities	Journal of Cybersecurity and Privacy (MDPI)	Oliveira et al	2026	Accuracy 94.23%, Precision 100%, F1 93.33%, detection within 25 seconds	Justifies the stage: Raw Logs → Preprocessing → Structured Events
[4]	Q-Cowrie: An Adaptive Honeypot to Analyses Attackers' Behavior	International Journal of Information Security (Springer)	Var Naseri et al	2026	. Develops an adaptive Cowrie-based honeypot for attacker behavior analysis	Justifies the honeypot layer and analysis of commands, sessions, and interaction patterns
[5]	Dynamic Cyber Deception Using AI-Driven Adaptive Honeypot Networks and Context-Aware Behavioural Analysis	Ain Shams Engineering Journal (Elsevier)	Alshehri	2026	Achieved 98% detection accuracy and 95% deception effectiveness	Defines a future direction: collect → analyze → understand → adapt
[6]	Knowledge Graph Visualization and Anomaly Detection From Cyber Threat Intelligence Dataset Using Graph Neural Networks	Computer Networks (Elsevier)	Yilmaz et al	2025	Graph SAGE F1 = 0.9721, GIN F1 = 0.9693, ROC-AUC > 0.99	Supports visualization of: Attacker → Target → Service → Attack → Technique

Ref.	Article Title	Source	Authors	Year	Main Findings	Relevance to Our Project
[7]	A Multi-Source Feature Fusion-Based Knowledge Graph Construction From Cyber Threat Intelligence to Facilitate APT Attribution in IDS	ISCIT (IEEE)	Au et al	2025	Builds a heterogeneous KG and links low-level alerts with APT information	Links: Attack → Technique → Threat Actor → Campaign
[8]	Multi-View Intrusion Detection Framework Using Deep Learning and Knowledge Graphs	Information (MDPI)	Li, Qiao, Lee	2025	F1 = 0.963 on TON_IoT and 0.960 in another setting	Proves that KG improves attack detection through relationship representation
[9]	Analyzing Cyber Attack Patterns and Detection Strategies Through T-Pot Honeypot Deployment	ICOSEC (IEEE)	Nair et al.	2025	Analyzed over 690,000 attacks, their patterns, and geographic distribution	Justifies using honeypots to collect: Attacker → Target → Port → Service → Pattern
[10]	Provenance-Based CTI Analysis with Knowledge Graph and RAG (KGRAG)	Cloud (IEEE)	cycloadd	2025	Threat actor identification = 95%, Provenance precision = 100%	Adds: Source → Evidence → Attack

3-2. Challenges Identified in Previous Studies

Despite the significant progress achieved by previous studies, they faced several challenges, some of which remain unresolved.

3-2.1First: Large Volume of Honeypot Data.

Honeypots can generate massive logs and attack sessions within short periods, as demonstrated by Nair et al. [9], who analyzed over 690,000 attacks. This volume makes manual analysis difficult, limiting analysts' ability to efficiently identify important attack patterns.

3-2.2 Second: Heterogeneous and Unstructured Data. Security information

is collected from diverse sources, such as honeypot logs, network traffic, CTI reports, and system events, making integration into a unified representation a major challenge. OntoLogX [1] indicated that ontology use can mitigate this issue but does not fully solve it.

3-2.3Third: Difficulty in Understanding Relationships Between Attacks.

Traditional systems analyze individual events separately, while a real attack consists of multiple related actions. Au et al. [7] showed that building a heterogeneous Knowledge Graph can help link these actions, but the process is complex and requires precise extraction mechanisms.

3-2.4 Fourth: High Complexity of Attacker Behavior.

Attackers may change their behavior during interaction, as shown by Q-Cowrie [4], making it difficult to identify the complete attack behavior from isolated events.

3-2.5 Fifth: Knowledge Graph Construction Complexity.

CTI-Thinker [2] confirmed that building a reliable Knowledge Graph from raw data requires accurate entity and relation identification, redundancy removal, and semantic consistency, which is a complex process requiring significant human andtechnical resources.

3-2.6 Sixth: Attribution and Evidence Interpretation.

KGRAG [10] indicated that threat actor identification is difficult because different attackers may use the same tools and techniques, necessitating clear distinction between observed evidence and-inferred-relationships.

3-2.7 Seventh: Real-Time Analysis.

Although Oliveira et al. [3] achieved detection within 25 seconds, many systems still analyze data after long delays, limiting their usefulness for rapid response.

3-2.8 Eighth: Explainability.

Alshehri [5] noted that complex systems may produce accurate classifications but limited explanations, making it difficult for analysts to understand the evidence behind an alert.

4-2. Practical Practices

Based on the literature review, several practical practices can be adopted in the present project.

4-2.1 First: Using Honeypots for Controlled Attack Data Collection

Nair et al. [9] demonstrated that honeypots provide realistic data on attackers. In the present project, Cowrie can be used in an isolated environment to collect data on commands, sessions, and attacker behavior, as done by Oliveira et al. [3] and Q-Cowrie [4].

4-2.2 Second: Normalizing and Preprocessing Logs Before KG Construction.

OntoLogX [1] emphasized cleaning raw logs and extracting important fields (timestamps, IP addresses, commands, services) into a consistent structure. This was applied by Oliveira [3] through deterministic preprocessing and session reconstruction.

4-2.3 Third: Representing Security Information as Entities and Relationships.

CTI-Thinker [2] and Au [7] demonstrated the possibility of representing relationships such as: Attacker → Targets → Service, and Command → Indicates → Technique, enabling deeper analysis than textual logs.

4-2.4 Fourth: Mapping Events to Known Attack Techniques.

OntoLogX [1] showed the possibility of linking observed evidence with MITRE ATT&CK tactics when sufficient evidence is available, helping analysts understand the techniques involved.

4-2.5 Fifth: Combining Multiple Sources of Evidence.

KGRAG [10] emphasized the importance of combining multiple observations (IP, login attempt, command, port, network activity) to obtain stronger evidence of malicious behavior.

4-2.6 Sixth: Visualization of Attack Relationships Using Graphs.

Yilmaz [6] showed that visualizing relationships between attackers, targets, services, and techniques helps analysts identify attack chains and recurring patterns.

4-2.7 Seventh: Maintaining Evidence and Traceability.

KGRAG [10] emphasized linking each relationship in the Knowledge Graph to the original log that generated it, providing traceability.

4-2.8 Eighth: Supporting Human Analysis.

Alshehri [5] noted that the system should assist analysts, not replace them, with automated analysis organizing data and identifying relationships while analysts validate results.

5-2.Conclusion

This chapter reviewed ten recent reference studies published during 2025 and 2026, addressing honeypots, cyber attack analysis, Cyber Threat Intelligence, and Knowledge Graphs. These studies showed that honeypots provide an effective method for collecting realistic information about malicious activities and attacker behavior [3], [4], [9], while Knowledge Graphs provide a structured approach for representing relationships among security entities [1], [2], [6], [7].

The studies also revealed several challenges, including the large volume of honeypot logs [9], heterogeneous data formats [1], difficulty in identifying relationships between events [7], real-time analysis requirements [3], Knowledge Graph construction complexity [2], attribution challenges [10], and the need for explainable analysis [5].

Chapter 3 : Developed System

1-3.Introduction

This chapter provides a detailed explanation of the methodology adopted in building the proposed system, which aims to overcome the structural limitations of traditional honeypots. The proposed solution integrates Cyber Deception techniques with Large Language Models (LLMs) and Knowledge Graphs within a multi-layered architecture. This chapter clarifies the overall system architecture, the protocols and tools used, and the data flow mechanism, starting from attack collection to extracting cyber knowledge and generating analytical reports.

2-3.Overall-System-Architecture

To ensure system flexibility and scalability, the proposed solution is designed based on a modular architecture divided into three interconnected main layers, as shown in Figure (1):

1.2-3 Collection Layer:

Responsible for enticing attackers and collecting raw data in an isolated environment.

2.2-3 Knowledge Layer:

Responsible for processing raw logs and transforming them into a structured Knowledge Graph.

3.2-3 Intelligence Layer:

Responsible for utilizing Retrieval-Augmented Generation (RAG) and contextual analysis to produce actionable security insights.

1.2-3.Collection-Layer:Honeypot-Deployment-and-Data-Collection

This layer is designed to operate within an isolated virtual environment (VPS) to ensure infrastructure security. The layer consists of two main components, as illustrated in Figure (2):

1- Traditional SSH Honeypot (Cowrie):

Used to emulate a Fake Shell and a Virtual File System to capture brute-force attacks and reconnaissance-commands.

2- LLM Honeypot: Based on the HoneyAI framework, where unfamiliar commands are forwarded to the LLM Engine to generate dynamic and realistic responses. This component features a Cache for common commands to minimize latency.
Every interaction (IP address, executed command, timestamp) is recorded via a unified Event Logger and saved in JSON format to facilitate automated processing in subsequent layers.

2.2-3.Knowledge-Layer:

This layer receives the Raw Logs and transforms them into Structured Knowledge using Natural Language Processing techniques. This layer consists of two main stages as shown in Figure (3):

1. LLM Analysis: Prompts are built containing the session and schema, allowing the model to extract Entities and Relations. These outputs are then validated (Validator) to ensure compliance with the utilized Ontology.

2. Graph Construction: Duplicate entities are merged (Entity Resolution), and commands are mapped to known attack techniques using the MITRE ATT&CK framework (Technique Mapping). Finally, these relationships are loaded into a graph database (Graph Loader) to form nodes, edges, and evidence.

3.2-3. Intelligence Layer: Retrieval-Augmented Generation (RAG)

To enhance analysis accuracy and overcome the limitations of pre-trained language models, a Retrieval-Augmented Generation (RAG) layer is integrated as shown in Figure (4). Upon a new query or session (Query Trigger), the Query Builder extracts entities, and the Retriever searches in two sources:

1. Knowledge Graph: To retrieve previous relationships and evidence.
2. Knowledge Base: Containing MITRE ATT&CK reports, known vulnerabilities (CVEs), and security reports.

The results are merged (Context Assembler) while maintaining provenance tracking, then passed to the LLM Generator to produce accurate, evidence-grounded contextual analysis.

4.2-3. Visualization and Reporting

To enable the security analyst to comprehend complex data, the final layer includes an interactive visualization interface as shown in Figure (5). This interface includes:
Graph Explorer: To visually display attack chains and relationships, Session Timeline: To review executed commands in chronological order Attack Statistics: To display the distribution of IP addresses, countries, and techniques. ,Report Generator: To produce evidence-based textual reports.

Figure 1 : proposed System Architecture

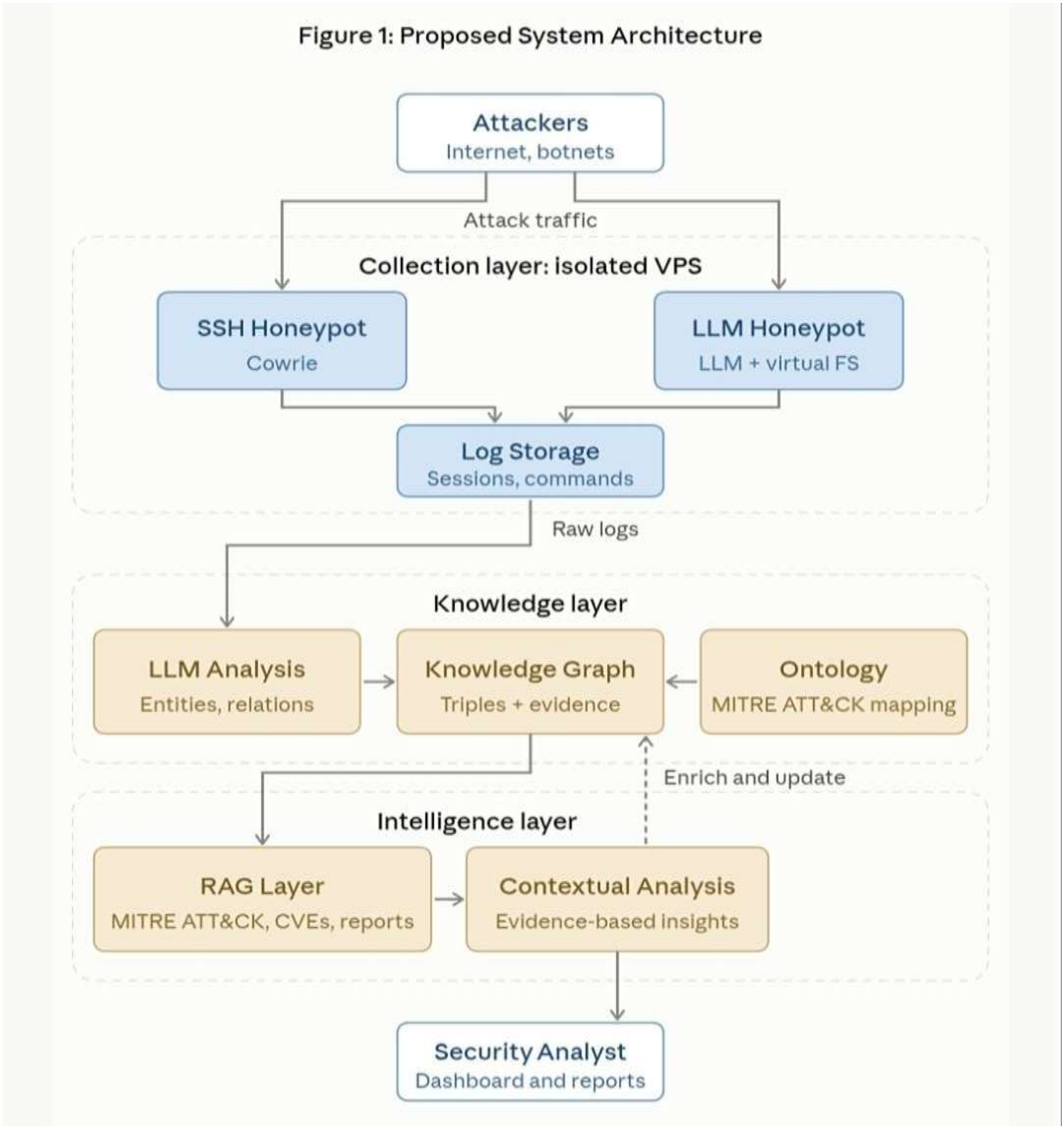


Figure 2 : Collection Layer

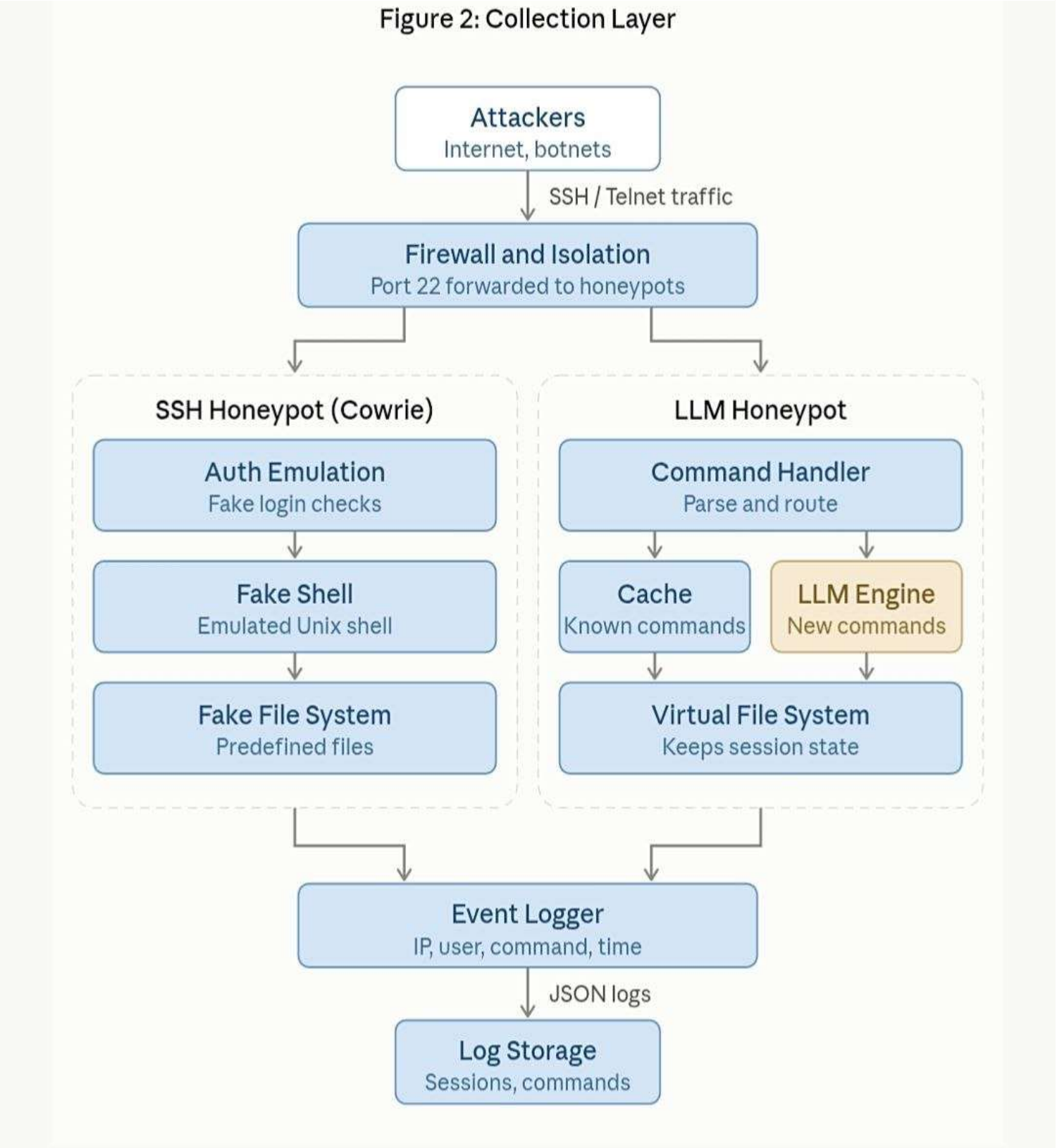


Figure 3: Knowledge Layer

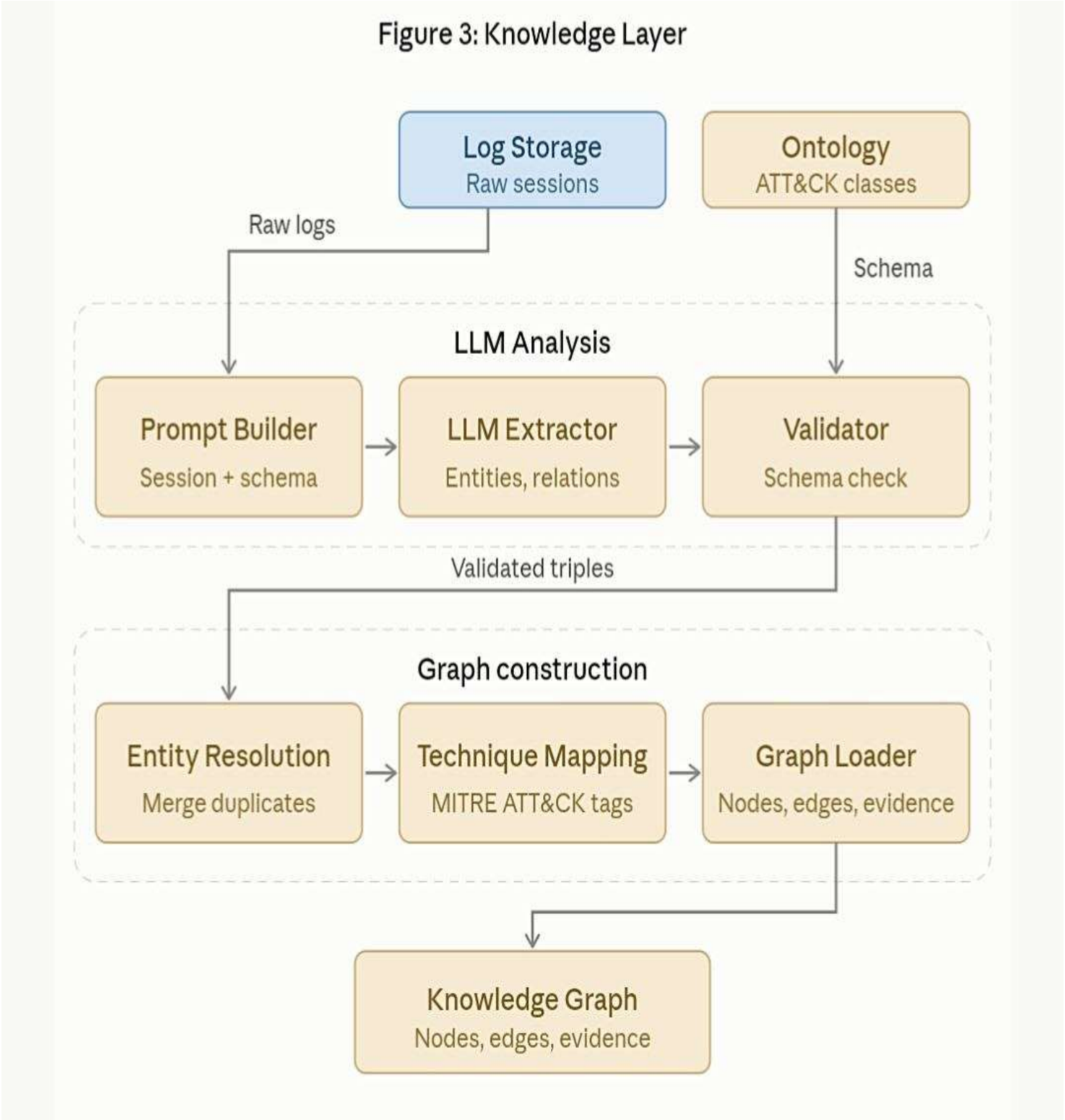


Figure 4: Intelligence Layer

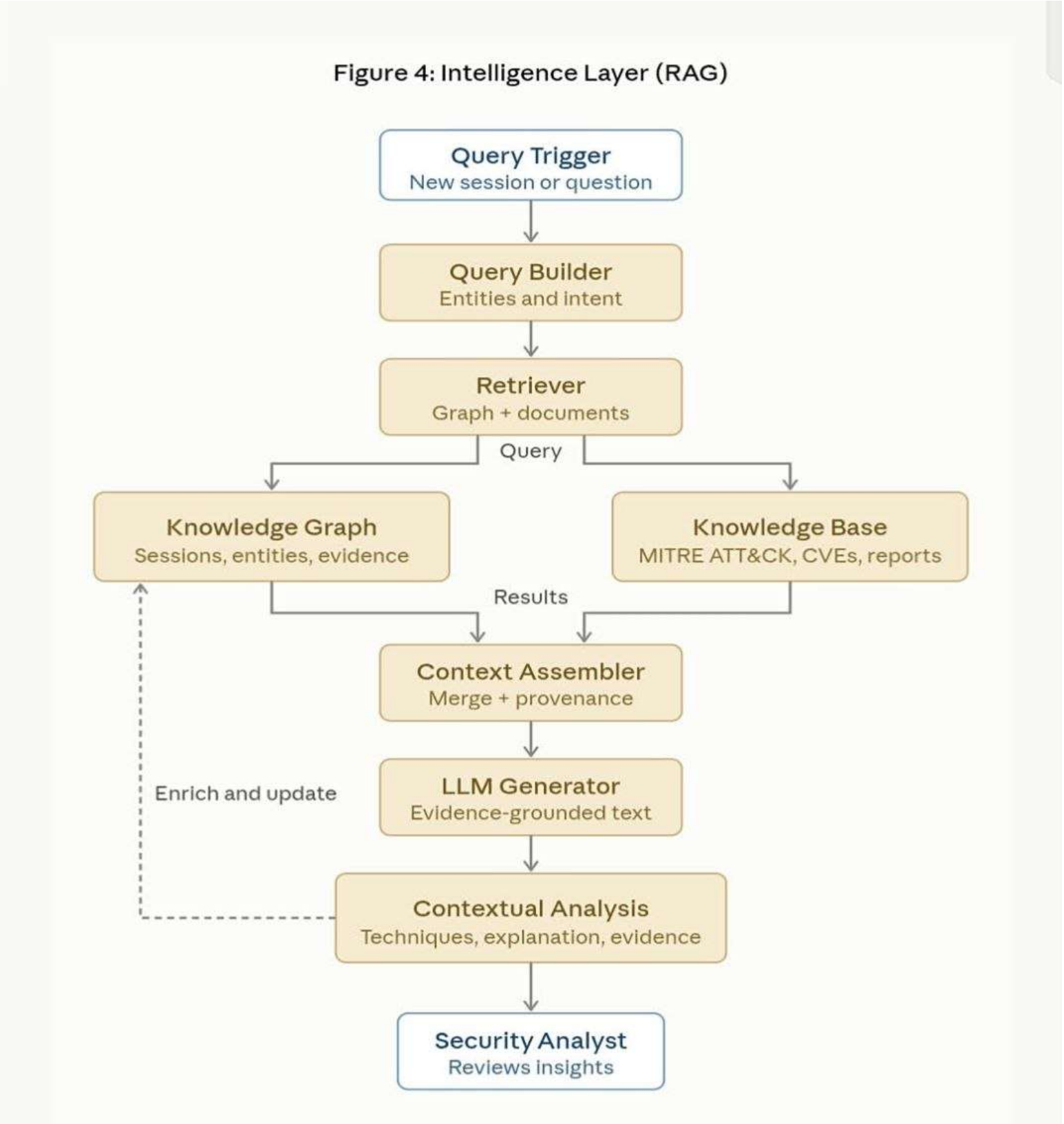
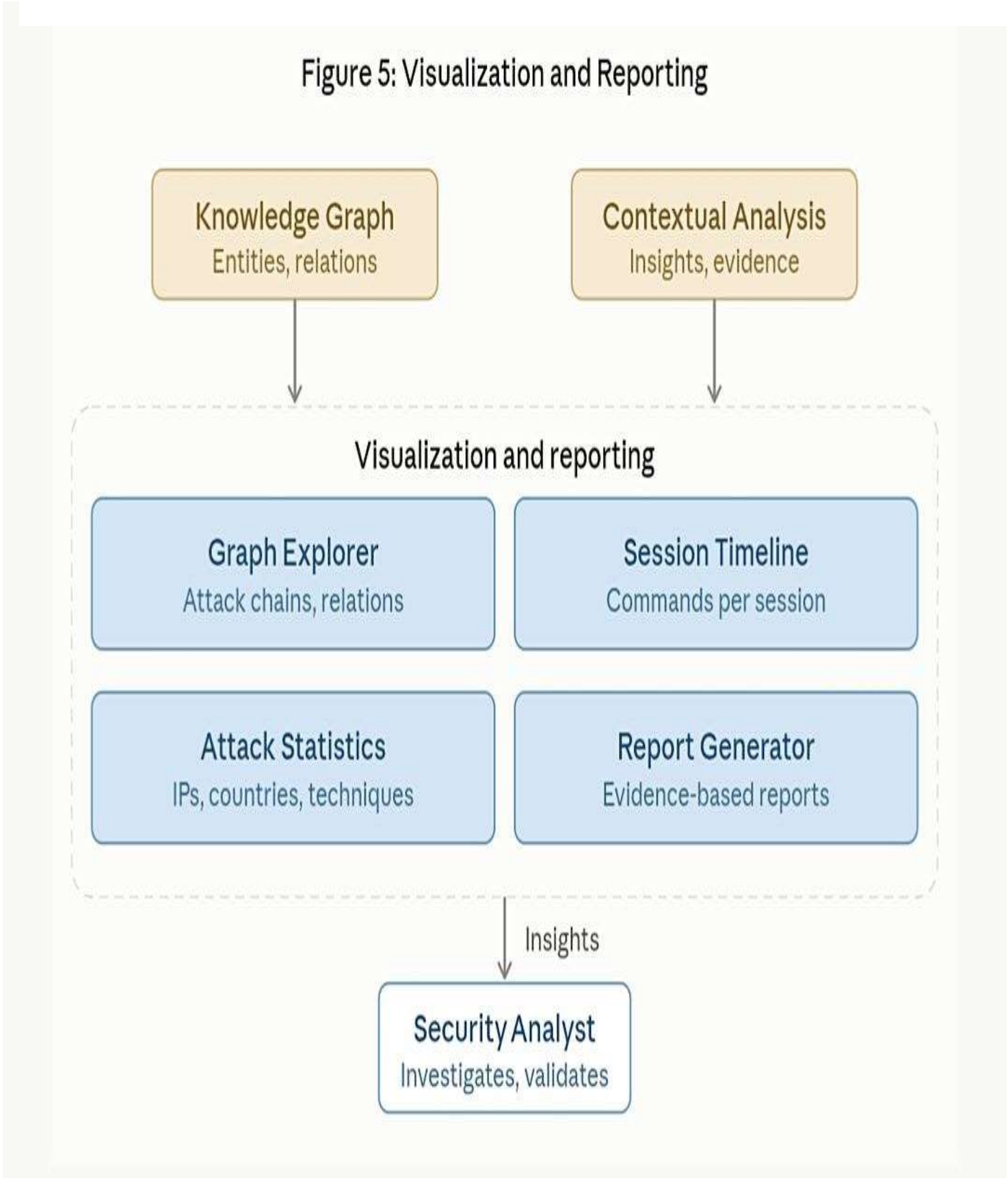


Figure 5: Victualing and Reporting



3-3.Conclusion

This chapter reviewed the comprehensive engineering methodology of the proposed solution, explaining the transition from merely collecting attack data via AI-enhanced SSH honeypots, to constructing a structured Knowledge Graph, and culminating in the use of RAG techniques to provide evidence-based contextual analysis. This multi-layered architecture ensures the achievement of the project's objectives in providing an accurate and explainable cyber attack analysis platform.

References

[1] L. Cotti, I. Drago, A. Rula, D. Bianchini, and F. Cerutti, "OntoLogX: Ontology-Guided Knowledge Graph Extraction From Cybersecurity Logs With Large Language Models," Advanced Intelligent Systems, vol. 8, no. 6, Art. no. e202501381, 2026.

[2] X. Yang, R. Zhong, Y. Chen, G. Peng, D. Yao, C. Chen, and C. Wang, "CTI-Thinker: An LLM-Driven System for CTI Knowledge Graph Construction and Attack Reasoning," Cybersecurity, vol. 9, Art. no. 106, 2026.

[3] R. Oliveira, T. Gomes, D. Pinho, and F. Almeida, "An Integrated Honeypot and LLM-Based Framework for Near Real-Time Detection and Behavioral Analysis of Malicious Activities," Journal of Cybersecurity and Privacy, vol. 6, no. 5, Art. no. 147, 2026.

[4] M. Var Naseri, I. Welch, J. Haseeb, and M. Mansoori, "Q-Cowrie: An Adaptive Honeypot to Analyse Attackers' Behaviour," International Journal of Information Security, vol. 25, Art. no. 60, 2026.

[5] M. Alshehri, "Dynamic Cyber Deception Using AI-Driven Adaptive Honeypot Networks and Context-Aware Behavioural Analysis," Ain Shams Engineering Journal, vol. 17, no. 4, Art. no. 104061, 2026.

[6] A. Yilmaz, R. Das, M. Ozdem, and B. Canberk, "Knowledge Graph Visualization and Anomaly Detection From Cyber Threat Intelligence Dataset Using Graph Neural Networks," Computer Networks, vol. 288, Art. no. 112632, 2025.

[7] T. T. M. Au, K.-K. Ngo, V.-H. Pham, and P. T. Duy, "A Multi-Source Feature Fusion-Based Knowledge Graph Construction From Cyber Threat Intelligence to Facilitate APT Attribution in IDS," in Proc. 24th Int. Symp. on Communications and Information Technologies (ISCIT), 2025.

[8] M. Li, Y. Qiao, and B. Lee, "Multi-View Intrusion Detection Framework Using Deep Learning and Knowledge Graphs," Information, vol. 16, no. 5, Art. no. 377, 2025.

[9] A. S. Nair, Mohana, J. S. Mathew, A. Chakkan, and A. S. Bhati, "Analyzing Cyber Attack Patterns and Detection Strategies Through T-Pot Honeypot Deployment," in Proc. 6th IEEE Int. Conf. on Smart Electronics and Communication (ICOSEC), 2025.

[10] "Provenance-Based CTI Analysis with Knowledge Graph and RAG (KGRAG)," in Proc. IEEE 12th Int. Conf. on Cyber Security and Cloud Computing (CSCloud), 2025.

[11] N. Ilg, D. Germek, P. Duplys, and M. Menth, "Beekeeper: Accelerating Honeypot Analysis With LLM-Driven Feedback," IEEE Access, vol. 13, pp. 1–18, 2025, doi: 10.1109/ACCESS.2025.361318.

[12] G. Adnyana, A. M. Dirgayusari, and K. J. Atmaja, "Data Visualization for Building a Cyber Attack Monitoring Dashboard Based on Honeypot Technology," Sinkron: Jurnal dan Penelitian Teknik Informatika, vol. 8, no. 4, pp. 2510–2520, Oct. 2024, doi: 10.33395/sinkron.v8i4.14144.

[13] P. Malhotra, "LLM Honey: A Real-Time SSH Honeypot with Large Language Model-Driven Dynamic Response Generation," Birla Institute of Technology and Science, Pilani, India, 2024.

[14] Y. Andrew, C. Lim, et al., "Knowledge Graphs for Cybersecurity: A Framework for Honeypot Data Analysis," in Proc. Int. Conf. on Informatics and Computing (ICIC), Bogor, Indonesia, Aug. 2023, pp. 1–6.

[15] Reworr and D. Volkov, "LLM Agent Honeypot: Monitoring AI Hacking Agents in the Wild," arXiv preprint arXiv:2410.13919, Jan. 2025.

[16] M. Sladic, V. Valeros, C. Catania, and S. Garcia, "LLM in the Shell: Generative Honeypots," in Proc. IEEE European Symposium on Security and

Privacy Workshops (EuroS&PW), Vienna, Austria, Jul. 2024, pp. 430–435, doi: 10.1109/EuroSPW61312.2024.00054.

[17] J. F. Cevallos-Moreno, A. Rizzardi, S. Sicari, and A. Coen-Porisini, "ACID: Beta-Testing Active Inference for Active Cyber-Defence," Journal of Reliable Intelligent Environments, vol. 12, no. 14, pp. 1–21, 2026, doi: 10.1007/s40860-026-00278-2.